SÃO PAULO
GOVERNO DO ESTADO
SÃO PAULO SÃO TODOS

Desenvolvimento WEB II
- Fundamentos de ECMAScript

Prof. Tiago Arruda

ECMAScript - Visão Geral

- JavaScript é a linguagem de programação que roda nos navegadores e servidores.
- ECMAScript é a especificação (o padrão) que define como a linguagem deve funcionar.
- Para React, focamos nas versões modernas (ES6+), que trouxeram clareza e poder.

ECMAScript - Visão Geral

- Browsers modernos suportam nativamente a maioria das features novas (ESModules, Async/Await).
- Node.js permite executar JS fora do navegador, essencial para ferramentas de build do React.
- Transpiladores (Babel) garantem compatibilidade com ambientes antigos quando necessário.

Sintaxe básica e tipos de dados

- Variáveis e Escopo:
  - Prefira const (imutável por referência) e let (mutável). Evite var para não poluir o escopo global.
  - Ponto e vírgula é opcional.
- Tipos Primitivos:
  - Imutáveis por natureza: string, number, boolean, null, undefined, symbol e bigint.
- Tipos de Referência:
  - Objetos e Arrays são mutáveis e armazenados por referência na memória.
- Truthy & Falsy:
  - Valores avaliados em booleanos. Falsy: false, 0, "", null, undefined, NaN. Todo o resto é Truthy.

```
// Variáveis modernas
const appName = "ReactApp";
let count = 0;
// Avaliação truthy/falsy
if (appName) {
  console.log("Exists!");
}
// Tipos primitivos
const isValid = true;
```

Objetos: estrutura e boas práticas

- Chaves e Acesso:
  - Coleções de pares chave-valor.
  - Use ponto (obj.key) para acesso direto ou colchetes (obj["key"]) para chaves dinâmicas.
- Atualização Imutável:
  - Nunca altere propriedades diretamente no state (obj.a = 1).
  - Crie um novo objeto copiando o antigo e sobrescrevendo o campo necessário.

```
// Objeto inicial
const user = { name: "Ana", age: 25 };
// Atualização segura (spread)
const updatedUser = {
  ...user, // Copia tudo de 'user'
  age: 26       // Sobrescreve 'age'
};
// Iterando chaves
Object.keys(user); // ["name", "age"]
```

Funções e Arrow Functions

- Arrow Functions (=>):
  - Sintaxe mais curta.
- Retorno Implícito:
  - Se o corpo tem uma única expressão, pode-se omitir chaves {} e return.
- Funções Puras:
  - Conceito chave: mesma entrada sempre gera mesma saída, sem efeitos colaterais.

```
// Clássica
function soma(a, b) {
return a + b;
}
// Arrow com retorno implícito
const dobro = n => n * 2;
// Arrow multilinhas
const processar = () => {
const x = 10;
return x + 5;
};
```

Desestruturação e Rest

- Extração Limpa:
  - Retire valores de objetos ou arrays e atribua a variáveis em uma única linha.
  - Reduz verbosidade como 'props.name'.
- Aliases e Padrões:
  - Renomeie variáveis durante a extração ({ a: novoNome }) e defina valores padrão se o campo não existir.
- Operador Rest (...):
  - Coleta "o resto" das propriedades não extraídas em um novo objeto/array.
  - Útil para separar props específicas de props repassadas.

```
// Objeto base
const person = { id: 1, name: "Leo",
role: "Admin" };
// Desestruturação + Rest
const { id, ...details } = person;
console.log(id);       // 1
console.log(details); // { name:
"Leo", role: "Admin" }
// Arrays
const [first, second] = [10, 20];
```

Spread Operator (...)

- Cópia Rasa (Shallow Copy):
  - O spread (...) expande os elementos de um iterável. Útil para criar cópias rápidas de objetos e arrays sem referenciar o original.
- Mesclar e Adicionar:
  - Combine múltiplos objetos ou arrays em um novo. Adicione novos itens preservando os existentes.
- Padrão em React:
  - Essencial para atualizar state complexo (ex: objetos aninhados), onde você copia o estado anterior e sobrescreve apenas o que mudou.

```
// Arrays: Copiar e Adicionar
const tasks = ["Ler"];
const updated = [...tasks, "Codar"];
// Objetos: Mesclar
const config = { theme: "dark" };
const userConfig = {
  ...config,    // Herda defaults
  lang: "pt-BR"// Adiciona novo
};
```

Map, Filter e Reduce

Map (Transformar)

Gera um NOVO array com o mesmo tamanho, aplicando uma função a cada item. Essencial para renderizar listas em JSX.

```
const nums = [1, 2, 3];
const dbl = nums.map(n => n * 2);
// [2, 4, 6]
```

Filter (Filtrar)

Gera um NOVO array apenas com elementos que satisfazem a condição (return true). Usado para remover itens.

```
const pares = nums.filter(n => n % 2 === 0);
// [2]
```

Reduce (Agregar)

Reduz o array a um único valor (número, objeto, etc). Poderoso, mas use com moderação para manter legibilidade.

```
const total = nums.reduce((acc, n) => acc + n, 0);
// 6
```

SÃO PAULO
GOVERNO DO ESTADO
SÃO PAULO SÃO TODOS

OBRIGADO